

ONIX Racing E03

Proyecto Web + AI Chatbot

STEM Racing Costa Rica · Supported by Honda

Documentación completa del proyecto — Explicación para el equipo

Generado: 26/5/2026, 8:46:25 p. m.

ÍNDICE

- 1. Estructura General (3 capas)
- 2. Frontend (lo que se ve)
- 3. Backend (el cerebro oculto)
- 4. API de DeepSeek (la IA)
- 5. Funcionalidades una por una
- 6. Seguridad
- 7. File Structure
- 8. Deploy (cómo está online)
- 9. Glosario de conceptos clave

1. ESTRUCTURA GENERAL

La página tiene 3 capas que trabajan juntas para que todo funcione:

Capa 1: Frontend (lo que se ve)

Es todo lo que aparece en el navegador cuando alguien entra al link. Incluye el HTML (el esqueleto de la página: textos, imágenes, botones), el CSS (los colores, animaciones, bordes, sombras, diseño responsive) y el JavaScript (todo lo que la página hace cuando interactuás: scrollear, apretar botones, cambiar idioma, reproducir el video, animar números).

El frontend vive en el archivo `onix_racing.html`. Ahí está TODO: el diseño, los textos, los scripts, los estilos.

Capa 2: Backend (el cerebro oculto)

Es un programa que corre en un servidor en la nube (Render). No se ve, pero hace todo el trabajo pesado: recibe los mensajes del chat y los manda a DeepSeek, guarda la telemetría, maneja el login del admin, limita peticiones para evitar ataques, sanitiza los inputs.

El backend vive en `server.js`, escrito en Node.js (JavaScript del lado del servidor).

Capa 3: API de DeepSeek (la inteligencia artificial)

DeepSeek es una empresa que tiene un modelo de lenguaje (como ChatGPT pero más barato y técnico). Le mandamos el contexto del equipo (archivo `context.txt`) más el mensaje del usuario, y DeepSeek devuelve la respuesta. Nosotros no entrenamos nada. Solo le damos contexto y ella responde en base a eso.

2. FRONTEND — EXPLICACIÓN DETALLADA

El frontend es todo lo que el usuario ve y con lo que interactúa. Está compuesto por:

HTML (HyperText Markup Language)

Es el lenguaje de marcado que define la estructura de la página. Cada elemento es una etiqueta: `<div>` para contenedores, `<section>` para secciones, `<button>` para botones, `<input>` para campos de texto, `<video>` para videos, `<img>` para imágenes. El navegador lee el HTML y muestra los elementos en pantalla.

CSS (Cascading Style Sheets)

Es el lenguaje que le da estilo a la página. Define colores (`--onix-purple: #411145`), fuentes (Bebas Neue para títulos, Syne para botones), tamaños, animaciones (`@keyframes`), diseño responsive (`@media` queries para adaptarse a celular), efectos de vidrio (`backdrop-filter: blur`), sombras, bordes, transiciones suaves.

JavaScript (JS)

Es el lenguaje de programación que hace que la página sea interactiva. Corre en el navegador del usuario. Ejemplos: cuando scrolleás y los elementos aparecen con animación (`IntersectionObserver`), cuando apretás el botón de idioma y todo cambia a español (`toggleLang`), cuando las imágenes del auto cambian solas (`setInterval`), cuando los números suben animados (`requestAnimationFrame`), cuando escribís en el chat y se envía el mensaje (`sendMessage`).

3. BACKEND — EXPLICACIÓN DETALLADA

El backend es un servidor Node.js con Express que corre en Render. No se ve, pero es el cerebro de la operación. Escucha en un puerto (3000) y espera peticiones HTTP.

Servidor Express

Express es un framework para Node.js que facilita crear rutas (endpoints). Cada ruta es una URL que el frontend llama para hacer algo específico.

Endpoints del backend

GET /	Sirve la página onix_racing.html
GET /admin	Sirve el panel admin.html
POST /api/chat	Recibe el mensaje, lo manda a DeepSeek, devuelve la respuesta
POST /api/telemetry	Recibe datos de telemetría y los guarda en telemetry.json
POST /api/admin/login	Verifica usuario/contraseña, devuelve un JWT
GET /api/admin/telemetry	Devuelve los datos de telemetría (requiere JWT)
POST /api/admin/logout	Elimina la cookie del token

Middleware

Son funciones que se ejecutan antes de llegar a las rutas. El backend usa: helmet() para headers de seguridad, express.json() para leer JSON, cookieParser() para manejar cookies, rateLimit() para control de peticiones, authRequired() para verificar el JWT del admin.

4. API DE DEEPSEEK (la inteligencia artificial)

DeepSeek es un modelo de lenguaje grande (LLM) similar a ChatGPT pero más económico y con buen rendimiento técnico. Su API funciona así:

Flujo de una conversación:

- El usuario escribe un mensaje en el chat del frontend
- El frontend hace un fetch POST a /api/chat con el texto y el idioma
- El backend recibe el mensaje y construye un system prompt: "Eres ONIX AI, asistente de STEM Racing Costa Rica E03. Responde en máximo 3 oraciones. Idioma: [en/es]. Contexto: [texto de context.txt]"
- El backend manda el system prompt + el mensaje del usuario a la API de DeepSeek
- DeepSeek procesa y devuelve la respuesta generada por IA
- El backend reenvía la respuesta al frontend
- El frontend muestra la respuesta en la ventana del chat

Parámetros de configuración:

Modelo	deepseek-chat
max_tokens	150 (respuestas cortas)
temperature	0.2 (respuestas factuales, ceñidas al contexto)
API Key	sk-89d689ff812646be97d3db01c74b3d28 (solo en backend)

5. FUNCIONALIDADES UNA POR UNA

Página Web

Hero

La pantalla principal al entrar. Tiene el logo ONIX con gradiente rosa-dorado, el subtítulo "Born from precision...", y dos botones: "Discover the Car" y "Chat with ONIX AI". Tiene animación de entrada (fadeUp).

Carrusel

Las imágenes del auto E03 que cambian solas cada 4.5 segundos. Hay dos slides con fotos reales del auto en base64. También se puede cambiar manualmente con los dots.

Marquee

Textos que se mueven horizontalmente infinitamente: PRECISION, SPEED, INNOVATION, TEAMWORK. Usan CSS animation con @keyframes marquee.

Stats animadas

4 tarjetas con números que suben animados cuando aparecen en pantalla: 6 miembros, ¡854K recaudados, +15 sponsors, 6,296 views. Usan IntersectionObserver + requestAnimationFrame.

Video cinemático

Un video MP4 (9.6 MB) que se reproduce automático en loop al scrollearlo. Tiene overlay con gradiente oscuro, glow rosa, y texto animado "Built to Perform".

Galería del auto

3 imágenes: una grande del E03 con label, y dos sub-imágenes con vistas trasera y frontal. Tienen hover effect con escala.

Identidad del equipo

4 tarjetas hexagonales con los símbolos: colibrí (velocidad), flor de Jamaica (naturaleza), piedra ónix (fuerza), hexágonos (6 miembros).

Equipo

6 tarjetas con iniciales, rol y nombre: Saria (Líder), Ivelisse (Patrocinios), Ian (Diseño), Sofia M. (Marketing), Sofia B. (Manufactura), Isabella (Gráfico).

Patrocinadores

12 badges: Honda, Coopecoceic, Dinos Pizza, El Rincón del Café, Namu, Tartaras, Retazos, IMP Grafika, El Shaddai, Monte Solís, Creston School, Dulce Tentación.

Misión y valores

4 value cards (Precisión, Creatividad, Trabajo en Equipo, Sostenibilidad) con visual de presupuesto y tarjetas de sostenibilidad.

Digital

3 metric cards con cifras de marketing: 6,296 vistas pico, 5,145 vistas promo, +76 seguidores en 48hrs.

Idioma

La página soporta inglés y español. Cada texto tiene su versión en ambos idiomas usando clases .txt-en y .txt-es. El CSS oculta/muestra según el atributo data-lang del HTML. La función toggleLang() cambia el idioma, actualiza la bandera y el placeholder del chat.

Chatbot

El asistente ONIX AI se activa desde cualquier parte de la página. Tiene un input de texto, botón de enviar, quick chips con preguntas predefinidas, indicador de escritura con 3 puntitos animados, burbujas de mensaje con estilos diferenciados (usuario vs bot), y timestamps. El historial se mantiene durante la sesión. Las respuestas son generadas por DeepSeek con el contexto del equipo.

Telemetría (anti-maliciosos)

Cuando alguien entra a la página, se ejecuta `captureTelemetry()` que captura:

- User-Agent: navegador, sistema operativo, tipo de dispositivo
- Geolocalización: coordenadas GPS (con permiso del usuario, fallback silencioso si rechaza)
- Canvas fingerprint: dibuja un texto único en un canvas invisible y genera un hash para identificar el navegador
- IP pública + ISP: consulta `ipapi.co`
- Honeypot: un input oculto que los bots llenan automáticamente. Si se detecta, se bloquea la IP

Todos los datos se envían a `POST /api/telemetry` y se guardan en `data/telemetry.json`.

Admin Panel (/admin)

Panel protegido con login. Usa autenticación JWT con cookie `HttpOnly` (el JS del navegador no puede leer el token). Una vez adentro, muestra una tabla con toda la telemetría recolectada: timestamp, IP, ISP, coordenadas, user-agent, fingerprint. Tiene filtros por IP y rango de fechas, paginación de 50 registros por página, y botón de logout.

6. SEGURIDAD

Rate Limiting (Control de peticiones)

El backend cuenta cuántas requests hace cada IP. Límites: 20 peticiones por minuto al chat (evita DoS y abuso de la API de DeepSeek), 5 intentos de login cada 15 minutos (evita fuerza bruta). Cuando se excede, devuelve error 429 "Too many requests".

Sanitización XSS (Limpieza de inputs)

Cualquier texto escrito en el chat pasa por la función `sanitize()` que convierte caracteres peligrosos (`<` `>` `&` `"` `'` `/`) en entidades HTML. Esto evita que un usuario malicioso pueda inyectar código JavaScript o HTML en el chat.

JWT HttpOnly (Tokens seguros)

Cuando el admin inicia sesión, el backend firma un JSON Web Token con una clave secreta (`JWT_SECRET`) y lo guarda en una cookie con la bandera `HttpOnly`. Esto significa que el JavaScript del navegador NO puede leer el token. Solo el servidor lo verifica en cada petición a `/api/admin/telemetry`. El token expira a las 4 horas.

Helmet (Headers HTTP seguros)

Helmet es un middleware que configura headers de seguridad: `X-Content-Type-Options: nosniff` (evita que el navegador interprete archivos como otro tipo), `X-Frame-Options: SAMEORIGIN` (evita clickjacking), `Referrer-Policy` (controla qué información se envía al hacer clic en links).

Honeypot (Trampa para bots)

Hay un input de texto oculto en la página (`left: -9999px, opacidad 0, height: 0`). Los humanos no lo ven ni lo llenan. Los bots automáticos sí. Si el backend recibe ese campo con contenido, agrega la IP a una lista negra (`blockedIPs`) y todas las peticiones futuras de esa IP son rechazadas con error 403.

Límite de caracteres

El backend rechaza mensajes de más de 2000 caracteres. El JSON body está limitado a 10KB. Esto evita ataques de buffer overflow y uso excesivo de recursos.

7. ESTRUCTURA DE ARCHIVOS

El proyecto tiene esta organización:

onix/ (carpeta raíz)

- server.js !' backend completo con Express, rutas, seguridad, JWT
- onix_racing.html !' página web completa (frontend: HTML+CSS+JS)
- admin.html !' panel de administración con login + tabla de telemetría
- package.json !' lista de dependencias (express, helmet, jsonwebtoken, etc.)
- .env !' API key de DeepSeek y configuración (NUNCA se sube a git)
- .gitignore !' excluye .env y node_modules
- render.yaml !' configuración para deploy automático en Render
- Dockerfile !' alternativa para deploy con Docker
- README.md !' documentación del proyecto
- data/context.txt !' texto con la información del equipo para la IA
- data/telemetry.json !' base de datos de telemetría (se crea sola)
- data/Video Project 4.mp4 !' video cinemático de la página

8. DEPLOY (cómo está online)

El proyecto está desplegado en Render, una plataforma cloud gratuita. El flujo es:

- El código se almacena en GitHub: github.com/DylanRamirezLopez/onix-racing
- Render está conectado al repositorio de GitHub
- Cada vez que se hace git push, Render detecta el cambio automáticamente
- Render ejecuta npm install para instalar dependencias
- Render ejecuta npm start (node server.js) para arrancar el servidor
- El servidor se despliega en la URL: <https://onix-racing.onrender.com>
- El panel admin está en: <https://onix-racing.onrender.com/admin>

Auto-Deploy:

Render tiene Auto-Deploy activado. Cada push a GitHub dispara un nuevo deploy automático en 1-2 minutos. No requiere intervención manual.

9. GLOSARIO DE CONCEPTOS CLAVE

Node.js

JavaScript que corre en el servidor, no en el navegador. Permite hacer backends con el mismo lenguaje que el frontend.

Express

Framework para crear servidores web en Node.js. Maneja rutas, middleware, peticiones HTTP.

API

Application Programming Interface. Una interfaz que permite que dos programas se comuniquen. Ej: nuestro backend habla con la API de DeepSeek.

Endpoint

Una ruta específica como /api/chat que el backend expone para recibir o enviar datos.

JSON

JavaScript Object Notation. Formato liviano para intercambiar datos, basado en pares clave-valor.

JWT

JSON Web Token. Un token cifrado que prueba que un usuario está autenticado. Contiene datos + firma digital.

HttpOnly Cookie

Cookie que el navegador guarda pero el JavaScript no puede leer. Solo el servidor la accede, evitando robos por XSS.

Rate Limit

Técnica que limita la cantidad de peticiones que un cliente puede hacer en un tiempo determinado.

Honeypot

Campo de formulario oculto que solo los bots detectan. Si se llena, se bloquea al remitente.

IntersectionObserver

API de JavaScript que detecta cuándo un elemento entra o sale del área visible del navegador.

Media Query

Regla CSS que aplica estilos diferentes según el tamaño de pantalla (@media max-width: 600px).

Deploy

Proceso de subir el código a un servidor para que sea accesible desde internet.

Auto-Deploy

Configuración que hace que el servidor se actualice automáticamente cuando hay cambios en el repositorio.

Middleware

Función en Express que se ejecuta entre la petición y la respuesta. Ej: autenticación, logging, rate limiting.

Sanitización

Proceso de limpiar datos de entrada para eliminar caracteres peligrosos como < > que podrían ejecutar código.

CORS

Cross-Origin Resource Sharing. Mecanismo que permite o bloquea peticiones entre diferentes dominios.

LLM

Large Language Model. Modelo de IA entrenado con grandes cantidades de texto para generar respuestas (DeepSeek, ChatGPT).

System Prompt

Instrucción inicial que se le da a un LLM para definir su comportamiento y contexto antes de recibir preguntas.

Canvas Fingerprint

Técnica que dibuja un texto/imagen único en un elemento canvas del navegador para generar un identificador único.

RequestAnimationFrame

Función de JavaScript que ejecuta código en cada frame del navegador (60 fps) para animaciones suaves.

ONIX Racing E03

Hecho con dedicación para el equipo

Documentación generada el 26 de mayo de 2026